言語処理系
Language Processing System

- ミドルパスにおける最適化
  - ループとその逐次最適化
  - 支配、支配木、バックエッジとナチュラルループ
  - データ依存解析
  - ループのベクトル化



早稲田大学理工学術院
情報理工学科
笠原博徳

2026年7月3日配布

(Control) Flow graphの例

```
B1  (1)   i := m-1
    (2)   j := n
    (3)   t1 := 4*n
    (4)   v := a[t1]
B2  (5)   i := i+1
    (6)   t2 := 4*i
    (7)   t3 := a[t2]
    (8)   if t3 < v goto (5)
B3  (9)   j := j-1
    (10)  t4 := 4*j
    (11)  t5 := a[t4]
    (12)  if t5 > v goto (9)
B4  (13)  if i >= j goto (23)
    (14)  t6 := 4*i
    (15)  x := a[t6]
B5  (16)  t7 := 4*i
    (17)  t8 := 4*j
    (18)  t9 := a[t8]
    (19)  a[t7] := t9
    (20)  t10 := 4*j
    (21)  a[t10] := x
    (22)  goto (5)
B6  (23)  t11 := 4*i
    (24)  x := a[t11]
    (25)  t12 := 4*i
    (26)  t13 := 4*n
    (27)  t14 := a[t13]
    (28)  a[t12] := t14
    (29)  t15 := 4*n
    (30)  a[t15] := x
```

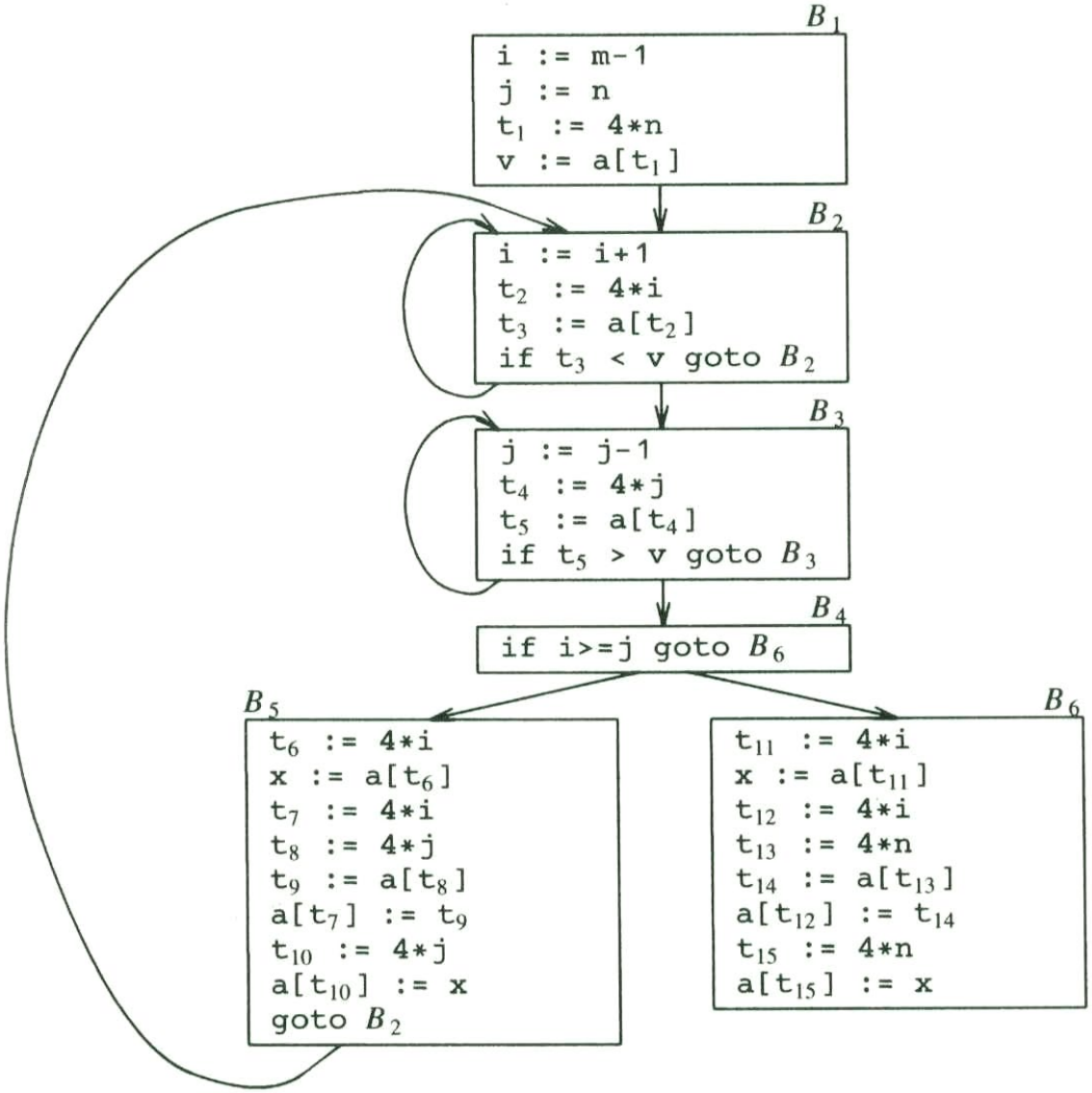


Fig. 10.5. Flow graph.

ループ

- フローグラフにおける以下のようなノードの集合
  - ループを構成するノード集合中の全てのノードは強連結である。
    - すなわちループ内では、任意のノードから任意のノードへ達する、長さ1以上のパスが存在する。
  - ループを構成するノード集合には入口が一つのみ存在する。
    - すなわちループ外のノードから、ループ内のノードに到達するために必ず通らねばならないノードが一つ存在する。
- 内部に他のループを含まない（それ以上ネストしない）ループをinnermost loopと呼ぶ。
  - 最外側ループ (Outermost Loop)
  - 最内側ループ (Innermost Loop)
- ループはプログラム実行時間の多くを消費する
  - ループの最適化（高速化）は実行時間短縮のため重要

ループ不変式のコードモーション
Code Motion of Loop Invariant

ループの中で、繰り返し同じ値を計算するのは無駄なので、その計算をループの外側に移動し、1回だけの計算に抑えることにより高速化する

```
while (I  <= limit-2 )
```



```
t =  limit -2

while (I  <=  t )
```

Strength reduction (強さの軽減) applied to 4 * j in block B_3

誘導変数(Induction Variable): ループの各繰り返しで一定値ずつ変化する変数

例 t_4 強さの軽減: 演算をコストの小さな演算に置き換える この例の場合 * から一

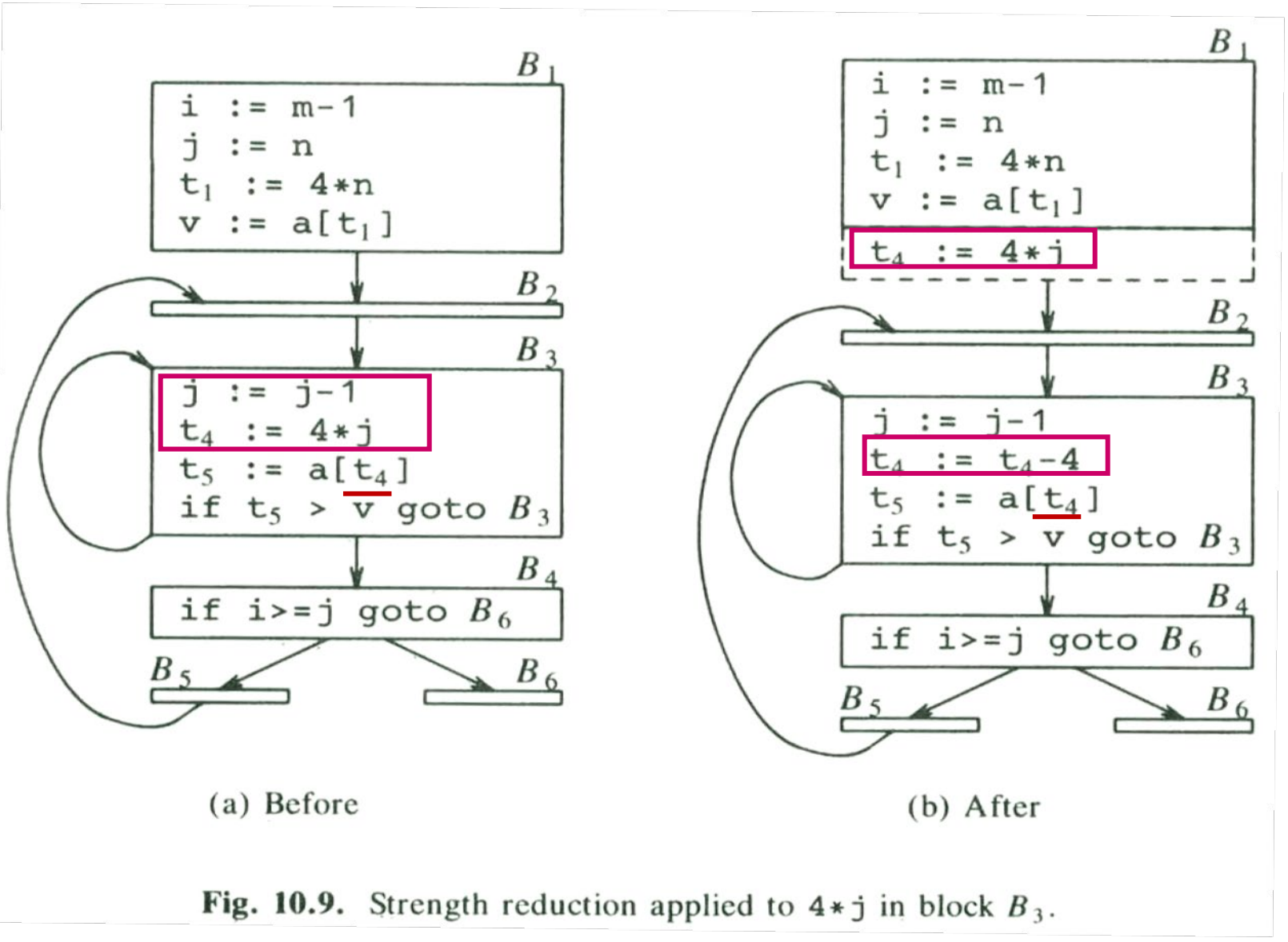


Fig. 10.9. Strength reduction applied to 4*j in block B_3.

Flow graph after induction-variable elimination

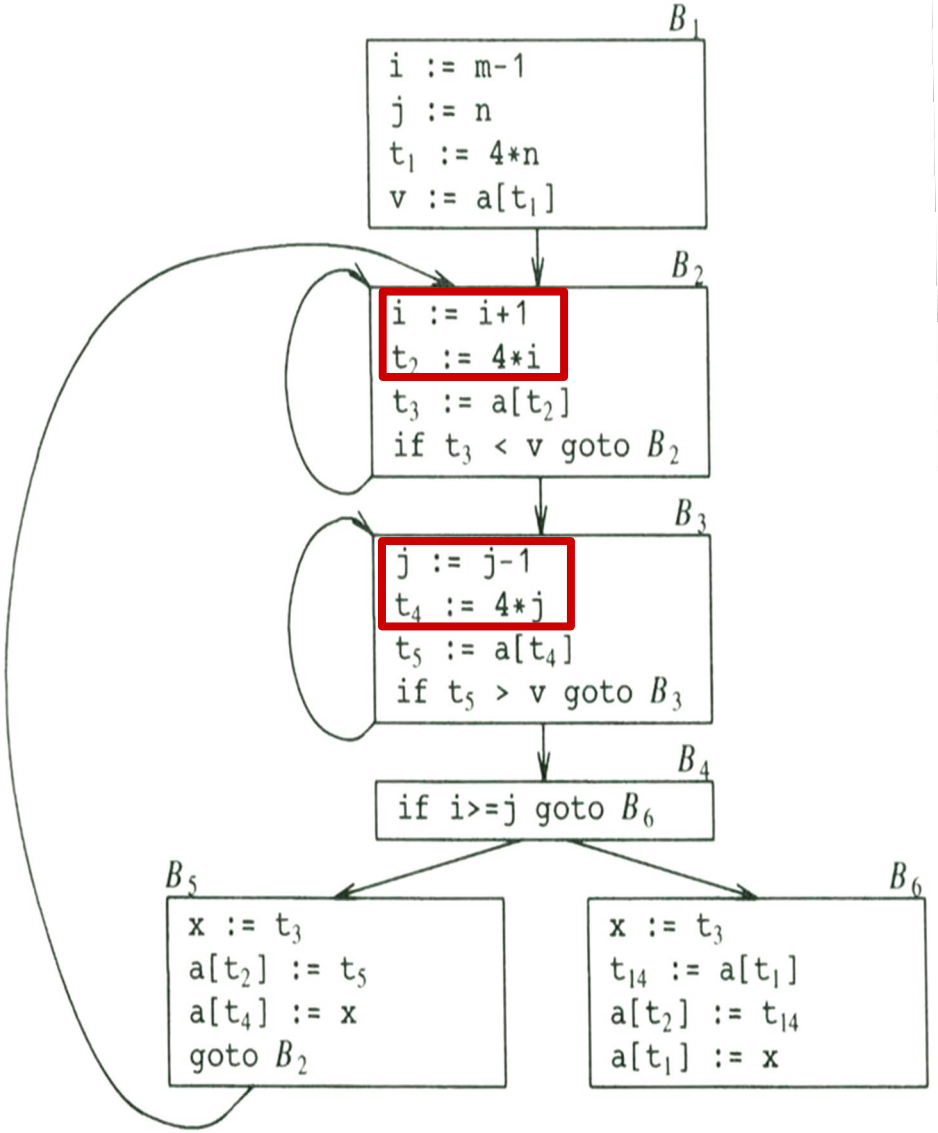


Fig. 10.7. B_5 and B_6 after common subexpression elimination.

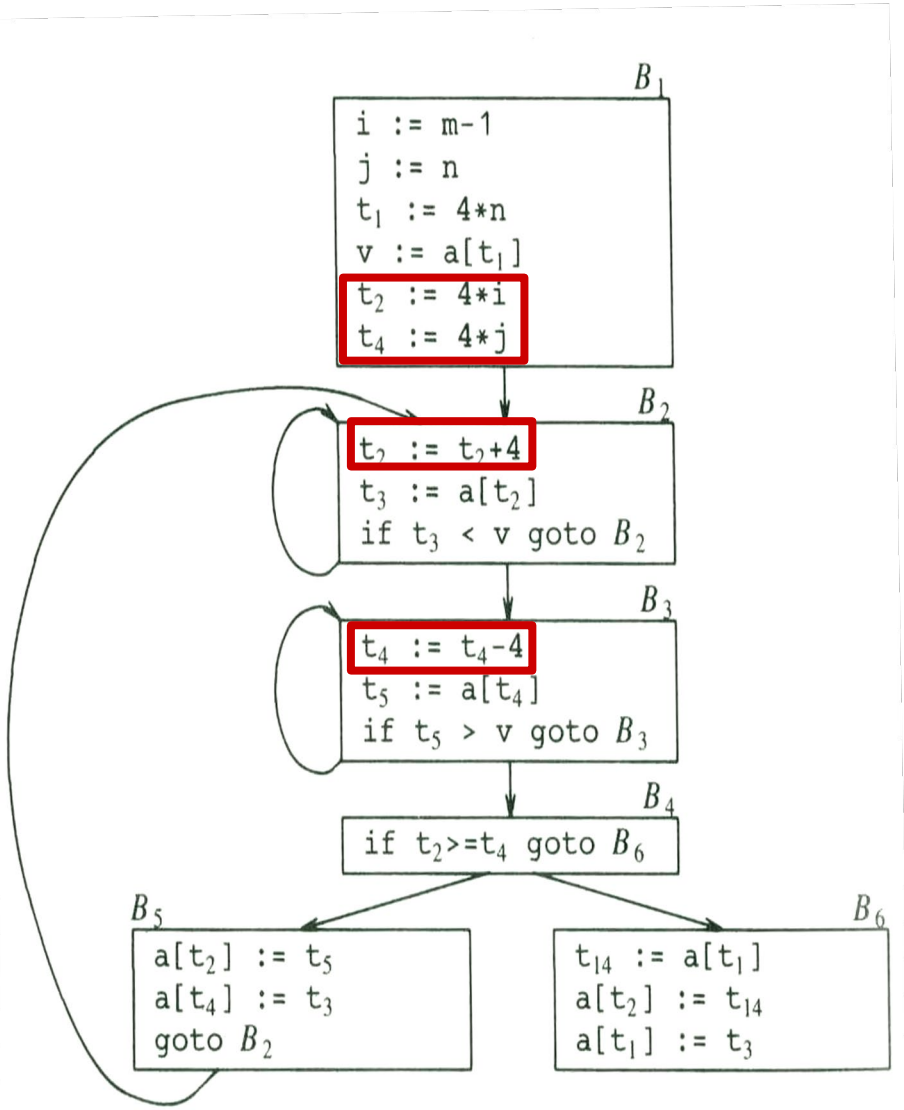


Fig. 10.10. Flow graph after induction-variable elimination.

Dominators (支配節)

フローグラフにおいて、開始ノードからあるノード *n* への任意の経路を考えたときに、どの経路をとっても必ずノード *d* を通れば、*d* は *n* を支配するといい、

d dom n

で表わす。
どのノードも自分自身を支配し、ループの入口ノードは、ループ中のノード全てを支配する。

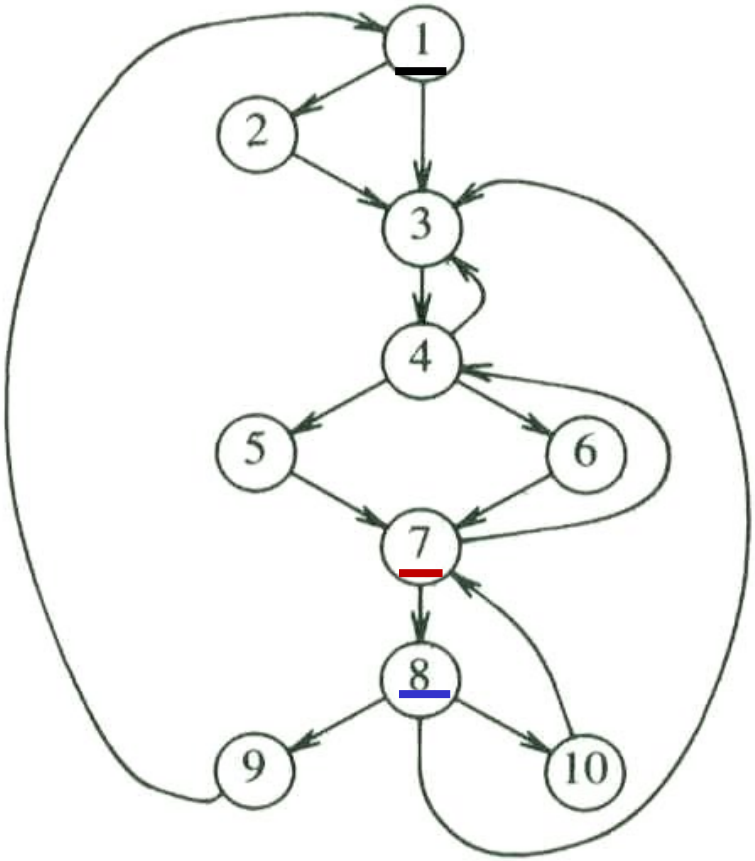


Fig. 10.13. Flow graph.

Dominator Tree (支配木)

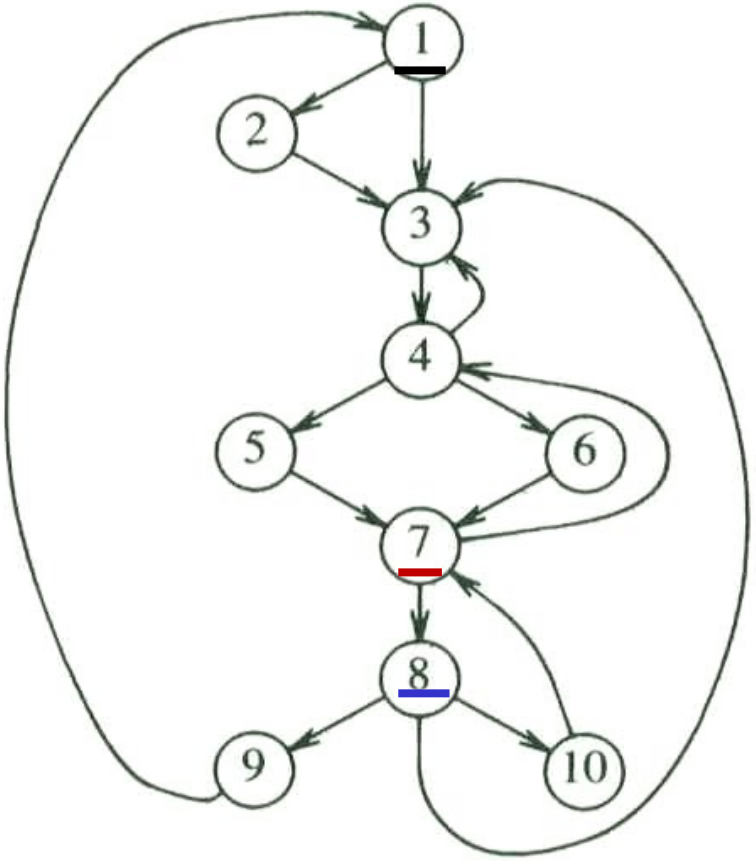


Fig. 10.13. Flow graph.

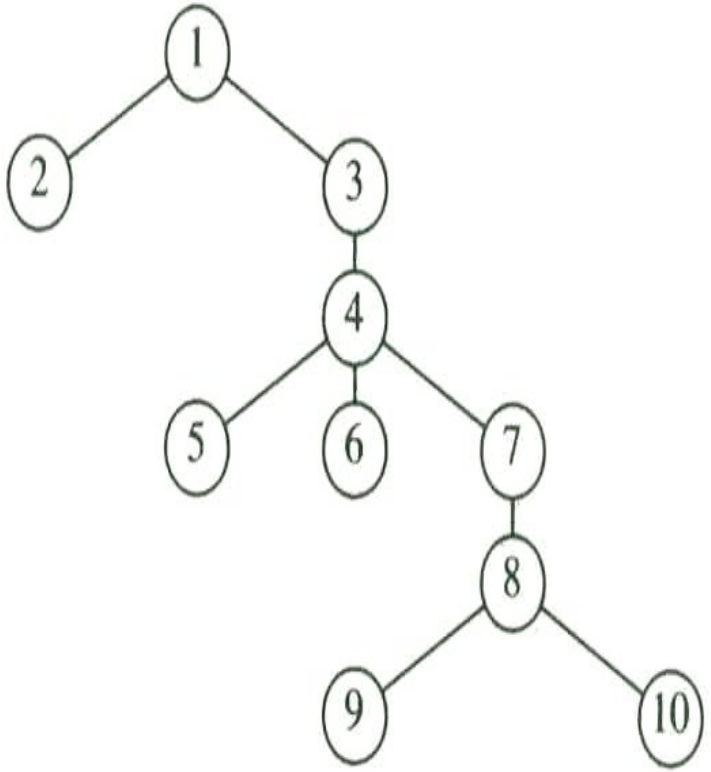


Fig. 10.14. Dominator tree for flow graph of Fig. 10.13.

•ノードnは、開始ノードからノードnまでの全てのパスで、ノードnを最後に支配する唯一の**直接支配ノード(Immediate Dominator)**を持つ。

•**ノードnの直接支配ノードm**は、**あるノードdに対し**、以下の性質を持つ。
(例えば、**m=7, n=8, d=1 の場合**)

d ≠n かつ d dom n なら d dom m （1≠8, 1 dom 8 なら 1 dom 7）

Natural (自然) Loop

＜Dominatorの応用例＞

フローグラフに対して、**Dominator(支配節)の性質を利用**すると、最適化の対象となる**ループを厳密に定義**できる。

ループは本質的に**次の2つの性質をもつ**。

1. ループは、“ヘッダー”とよぶ入口点をただ1つ持つ。
   このヘッダーはループ中のすべての節点を支配する。
2. ループの繰返し方は少なくとも1通りなくてはならない。
   ループにはヘッダーへ戻るパスが少なくても1つは存在しなければならない。

<Natural Loop の見つけ方>

フローグラフの中から**ループをすべて見つけ出す**には、
グラフ中でエッジの終点が、エッジの始点を支配するような辺（Back Edge：**後向きの辺**）を見つければよい。

（エッジ $a \rightarrow b$ において、aを**始点**、bを**終点**という。）

Natural Loopの例

・Back Edge (バックエッジ)
図10.13において、
辺7→4を考えると、4 *dom* 7である。
同様に、辺10→7も、7 *dom* 10である。
これと同じ性質をもつ辺には、他に4→3、8→3、
および9→1がある。
これらの辺はバックエッジ(Back Edge)であるから、
フローグラフの中でループを形成する辺となる。

・Natural Loop （ナチュラル・ループ）
バックエッジ $n \rightarrow d$ が与えられたとき、
d を通らないで n に到達できる節点の集合に
d を加えたものを、そのバックエッジに構成される
Natural Loop(ナチュラル・ループ)
と言う。 節点 d はループのヘッダーである。

図10.13において、辺10→7のナチュラルループを
考えると、節点7を通らないで10に到達できる節
点は、8と10しかないので、そのループは節点7,8
および10からなる。9→1で形成される
ナチュラル・ループはこのフローグラフ全体である
(10→7 →8 →9のパスが存在するため)。

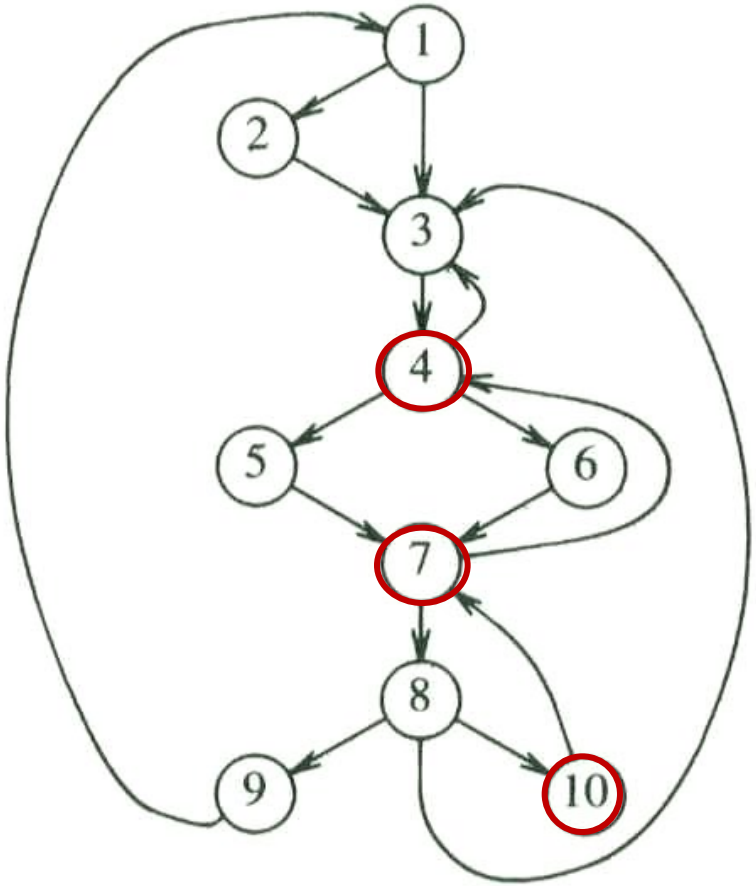


Fig. 10.13. Flow graph.

10→7: {7,8,10}
7→4: { 演習 }
4→3, 8→3: {3,4,5,6,7,8,10}

Reducible Flow Graph (可約フローグラフ)

“可約フローグラフ” の性質:

- ループの外部から途中へのジャンプがない。
- ループの入口はヘッダー以外にはない。

実際のプログラムから作られるフローグラフは、通常、下に定義する可約フローグラフという種類に属す。

- If-then-else文(条件分岐)、while-do文(ループ)、逐次、といった構造化された制御文を使用するかぎり、プログラムのフローグラフはつねに可約である (構造化定理)。
- goto文をあっても、ほとんどのプログラムは可約。

可約のフローグラフの定義:

フローグラフGの辺が、前向きの辺と後向きの辺とよぶ互いに独立なグループに分割できれば、そのグラフは**可約**であるという。辺は次の性質をもつ。

1. 前向きの辺のグループは、Gの開始節から、どの節点にも到達できる閉路のないグラフを形成する。
2. 後向きの辺のグループは、終点が始点を支配する辺(バックエッジ)だけからなる。

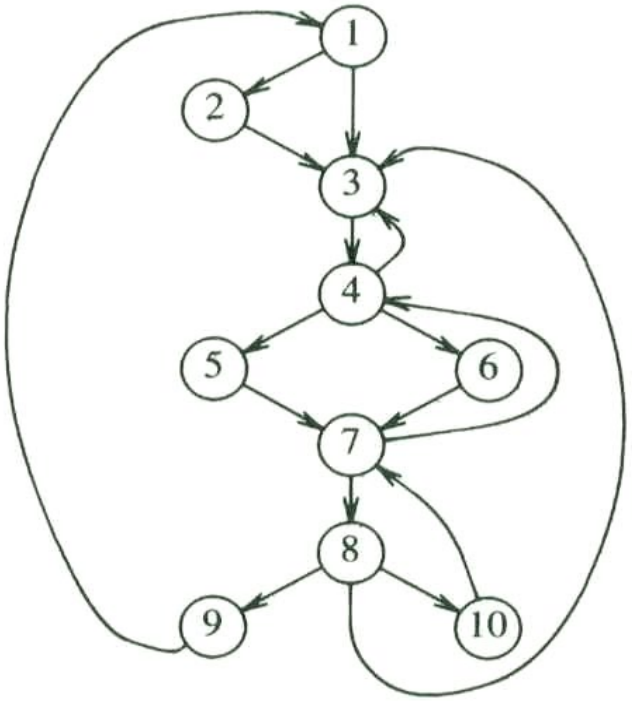
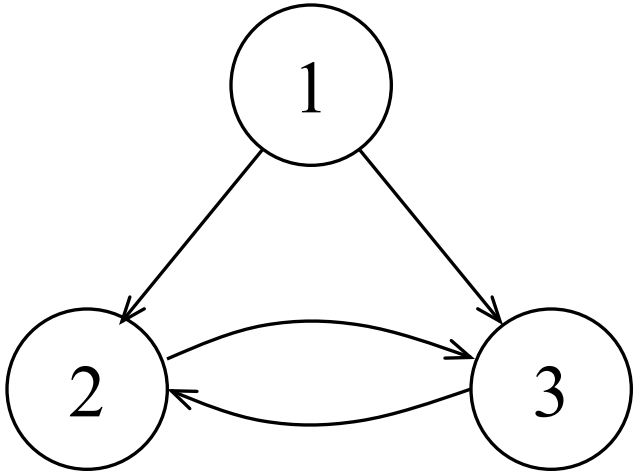


Fig. 10.13. Flow graph.

バックの辺 4→3, 7→4,8→3, 9→1, 10→7 を取り除いたグラフ(前向きの辺のみからなる)には閉路がないのでこのグラフは可約。

Non-reducible Flow Graph　非可約フローグラフ



ノード2と3は、ループを構成しているがナチュラルループを構成するバックエッジ（終点が始点を支配するバックエッジ）がない。

また、可約であれば閉路のない前向きのエッジだけであるべきだが、閉路を形成しているので可約ではない。

また、ループは2と3の二つのヘッダーをもつ。

並列処理方式の種類

Flynn's Classification of Parallel Processing Systems

表1.1　並列処理方式の種類　(Flynnの分類)

SISD (Single Instruction stream Single Data stream)		
1.命令パイプライン	Instruction Pipelining	（細粒度:Fine grain）
2.複数演算ユニット＊	Multiple Operation Units	（細粒度: Fine grain）
3.スーパースカラ＊	Superscalar	（細粒度: Fine grain）
4.VLIW (Very Long Instruction Word)＊		（細粒度: Fine grain）
SIMD (Single Instruction stream Multiple Data stream)		
5.演算パイプライン（ベクトル　プロセッサ）＊	Vector Processor	（細粒度: Fine grain）
6.プロセッサアレイ	Processor Array	（細粒度: Fine grain）
7.シストリックアレイ＊	Systolic Array	（近細粒度: Near Fine grain）
MIMD (Multiple Instruction stream Multiple Data stream)		
8.マルチプロセッサ	Multiprocessor	（近細、中、粗粒度: Near Fine, Medium,, Coarse grain）
9.データフローマシン（非ノイマン形コンピュータ）	Data Flow Machine: Non-von-Neumann（細粒度）	

＊　他の分類とみなすこともできることを示す: * shows it can be classified into another type.

Stanford Univ. Prof. Michael Flynn

- **IBM 7090 and 7094/II**
- **System 360 Model 91 Central Processing Unit**

2015 IEEE Computer Pioneer Award
1995 IEEE Harry H. Goode Memorial Award
1992 ACM/IEEE Eckert-Mauchly Award





Computer Science GATES BLDG



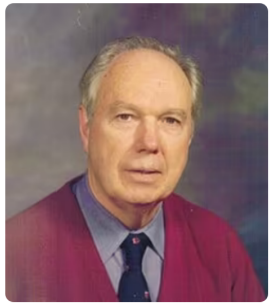
Home / Publications / Tech News / Insider Membership News

In Memoriam: Remembering Mike Flynn

passed away peacefully December 24, 2025

By Ruby B. Lee, Charlie Neuhauser, and Timothy M. Pinkston, former Flynn Ph.D. students on January 6, 2026

https://www.computer.org/publications/tech-news/insider-membership-news/in-memoriam-mike-flynn



Michael J. Flynn is a widely respected contributor—indeed a *giant*—in the field of Computer Architecture. He made highly significant and impactful contributions throughout his career, both in industry and in academia. Sadly, he passed away peacefully December 24, 2025, having lived a long and full life.

Born May 20, 1934, in New York, NY, Flynn earned his Bachelor's, Master's, and Ph.D. degrees in Electrical Engineering from Manhattan College (1955), Syracuse University (1960), and Purdue University (1961), respectively, and he received an honorary Doctor of Science degree from the University of Dublin (1998). After ten years as a design engineer and project manager at IBM (1955-65, in Endicott and Poughkeepsie, NY), he became a member of the faculty at the University of Illinois at Chicago (1965-1966), Northwestern University (1966-1970), and Johns Hopkins University (1970-1975) before joining Stanford University in 1975 as Professor of Electrical Engineering. He taught internationally, in Ireland, other places in Europe, Singapore, and Japan.

プロセッサアレイ (Processor Array)

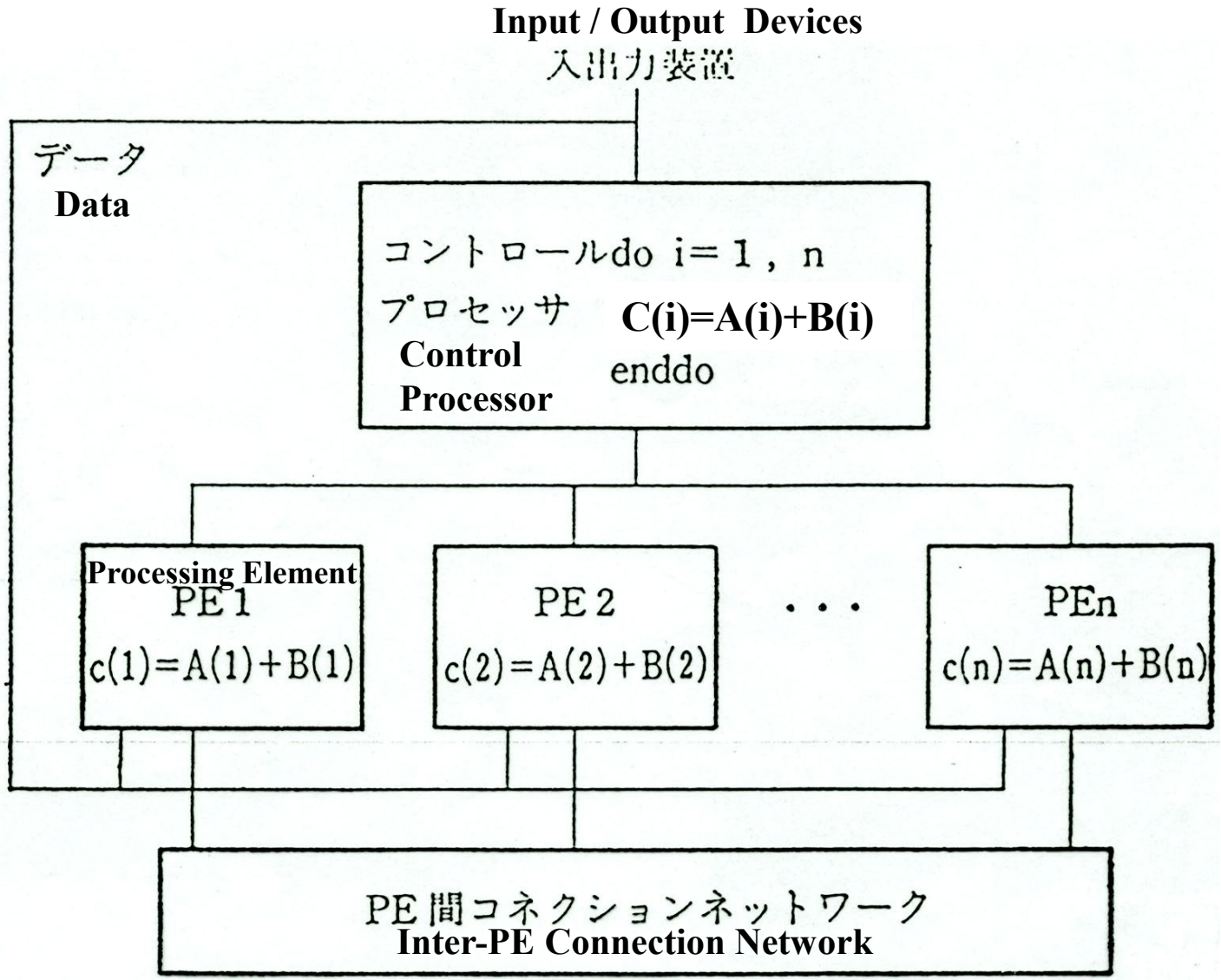


図 1.9 プロセッサアレイ

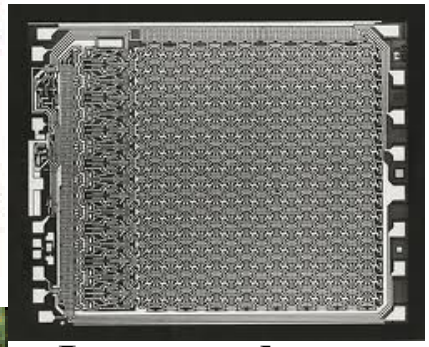
ILLIAC IV, Univ. Illinois at Urbana-Champaign & Burroughs



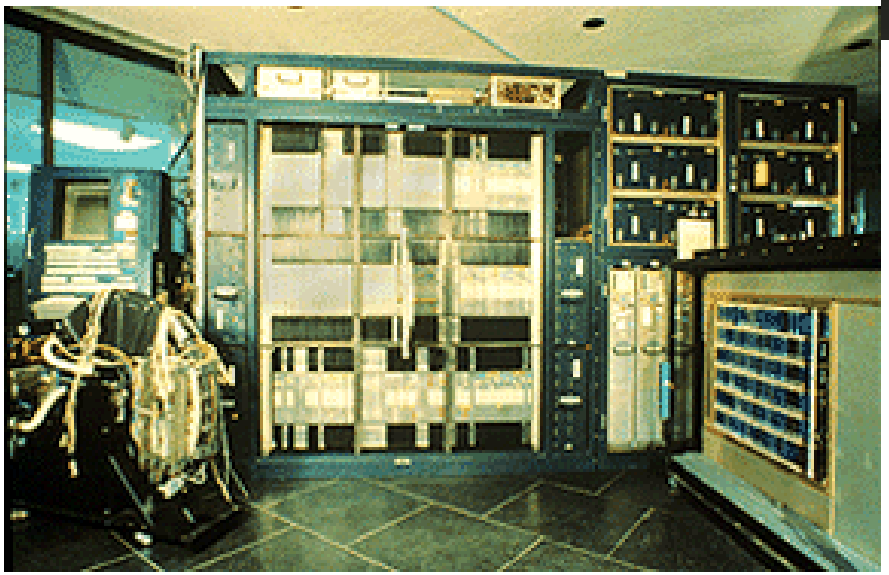


SIMD
64 Processor Element, Processor Array

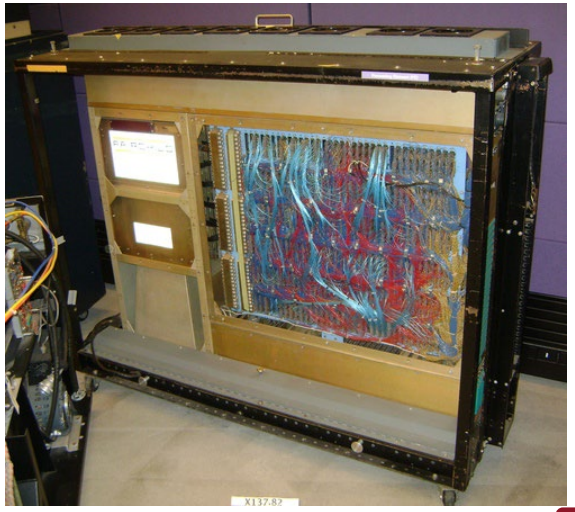
1972-3, NASA
200 MIPS,
300 MOPS,
1 billion bits per second of I/O transfer



Integrated circuits



Processing Element

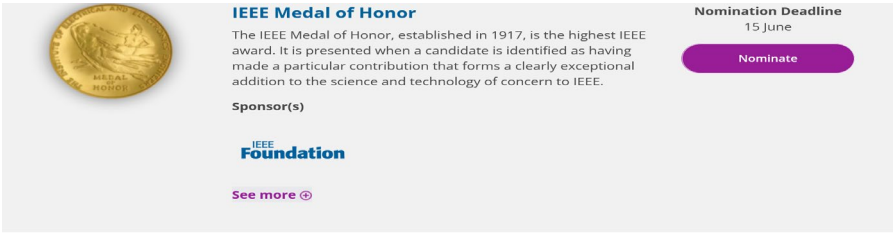


IEEE Awards
@IEEEAwards

Congratulations to @IllinoisCS' David J. Kuck on his 2024 @IEEEorg Frances E. Allen Medal, sponsored by @IBM, for pioneering work in vector and #ParallelComputer architecture, s#oftware, and compilers that enables many performance-sensitive applications: bit.ly/IEEEAwards-Rec...

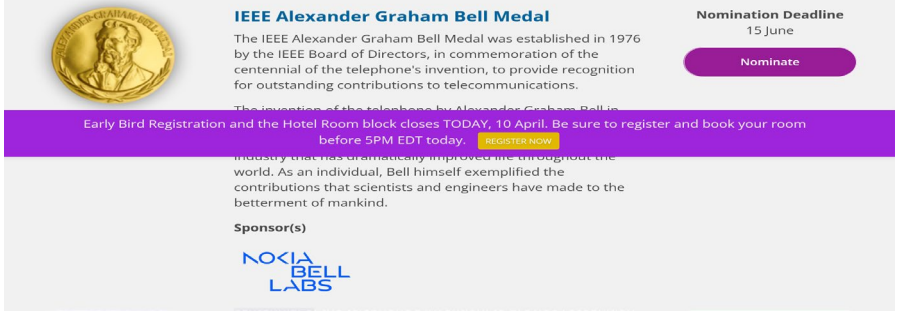
ポストを翻訳

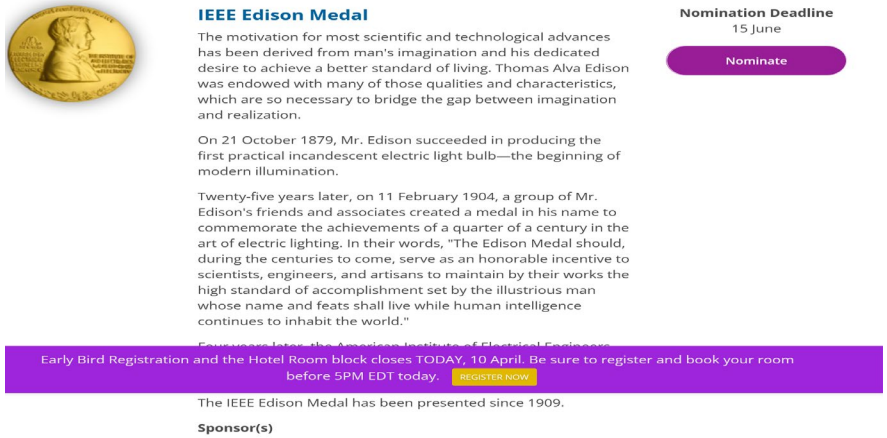






2026: CAPTCHA, Duolingo





https://corporate-awards.ieee.org/corporate-awards/

ILLIAC IV における PE 接続方式

ILLIAC IV Inter-PE Connection Network

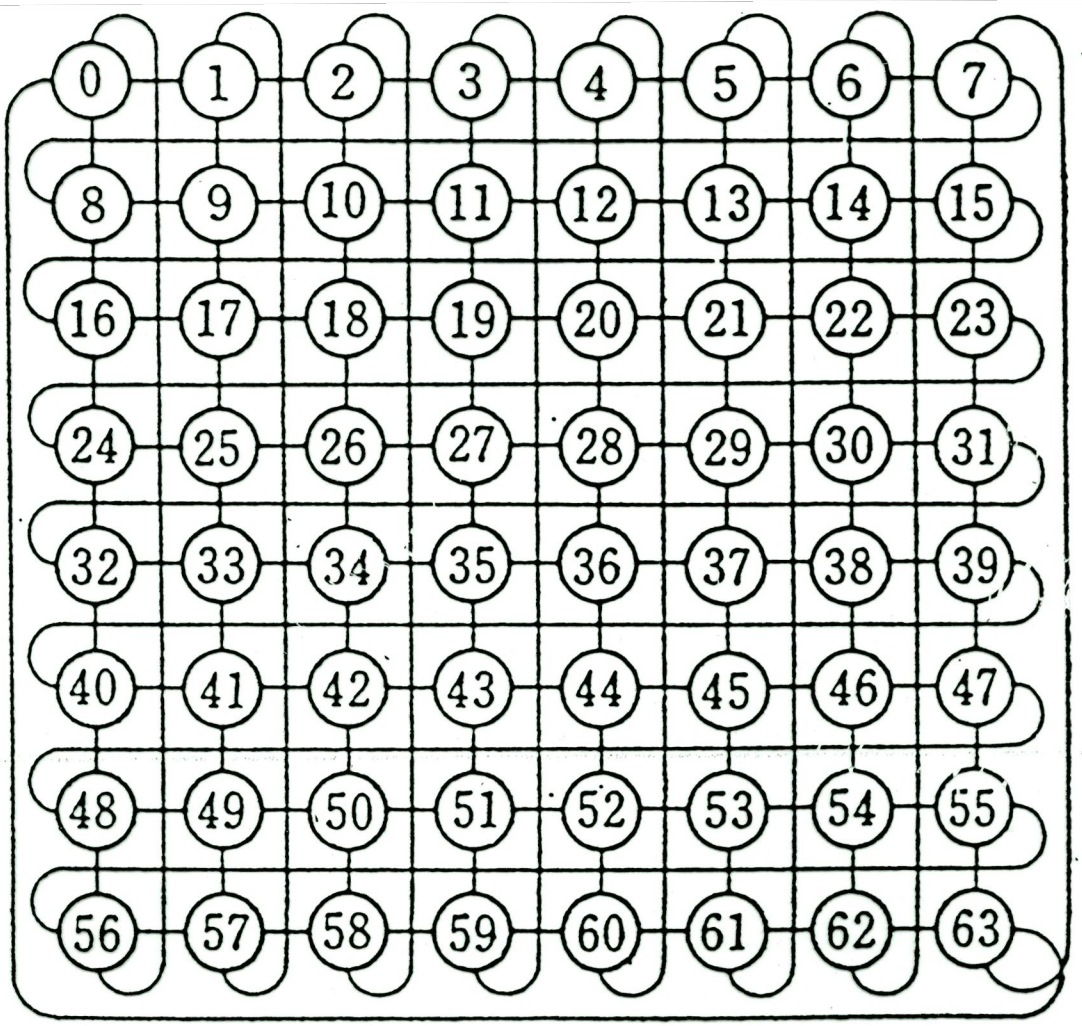


図 1.10 ILLIAC IV における PE 接続方式





Prof. David J Kuck (Univ. Illinois, Intel)
IEEE Computer Pioneer Award 2011
Ms. Diane B. Greene (VMware Cofounder & CEO)
IEEE Computer Society Computer Entrepreneur Award 2011





2018.03.08 WasedaUniv. Symposium on Future of High Performance Green Computing 2018 (HPGC2018)



MPP System Architecture (Goodyear Aerospace Satellite Image Processing Application)

320M byte/s

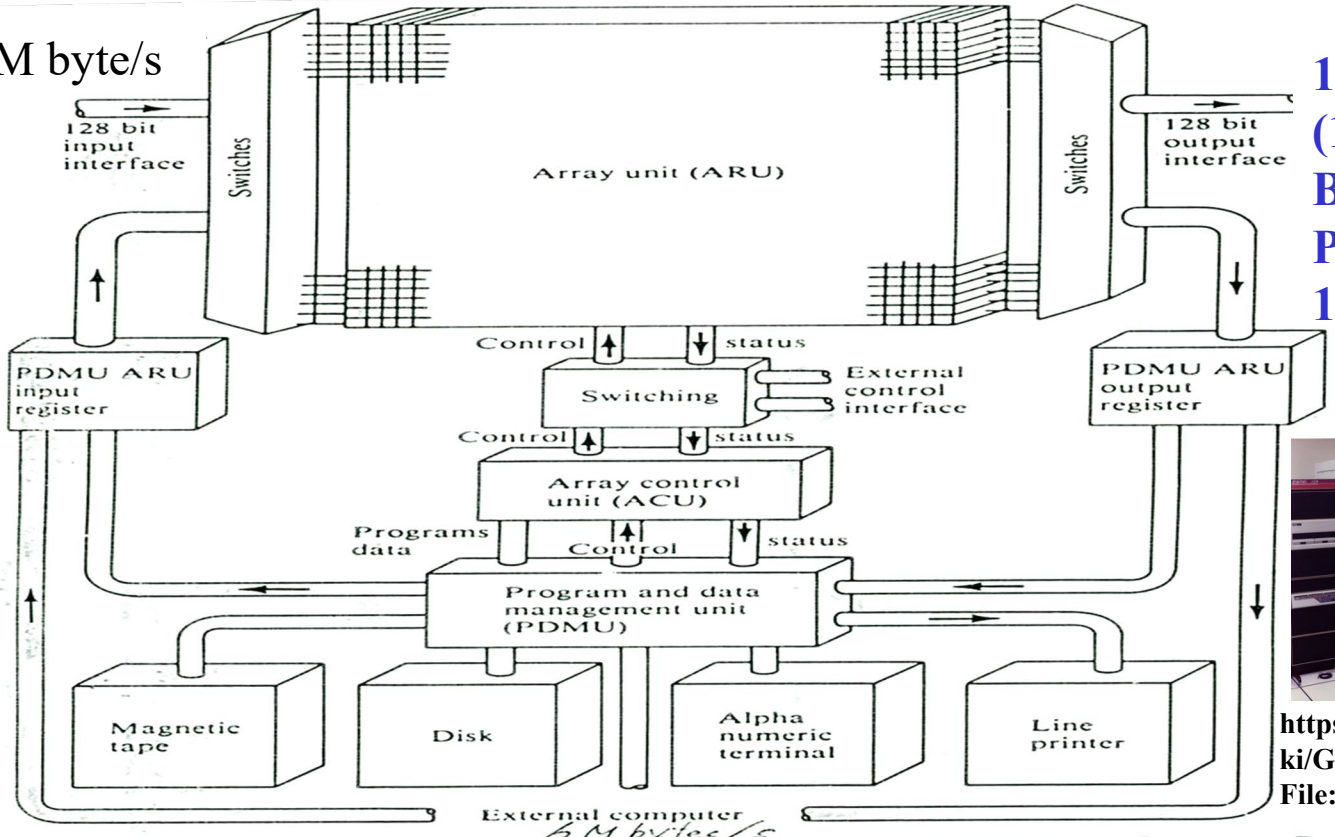


Figure 6.16 The system architecture of the MPP system. (Courtesy of *IEEE Trans. Computers*, Batcher, 1980.)

16,384 PEs
(128 *128)
Bit slice
Per clock
10MHz

Developed
1979-1983,
Used:
1985-1991



https://en.wikipedia.org/wiki/Goodyear_MPP#/media/File:Goodyear_MPP.jpg

	Peak speed (mops*)
Addition of arrays	
8-bit integers (9-bit sum)	6553
12-bit integers (13-bit sum)	4428
32-bit floating-point numbers	430
Multiplication of arrays (element-by-element)	
8-bit integers (16-bit product)	1861
12-bit integers (24-bit product)	910
32-bit floating-point numbers	216
Multiplication of array by scalar	
8-bit integers (16-bit product)	2340
12-bit integers (24-bit product)	1260
32-bit floating-point numbers	373

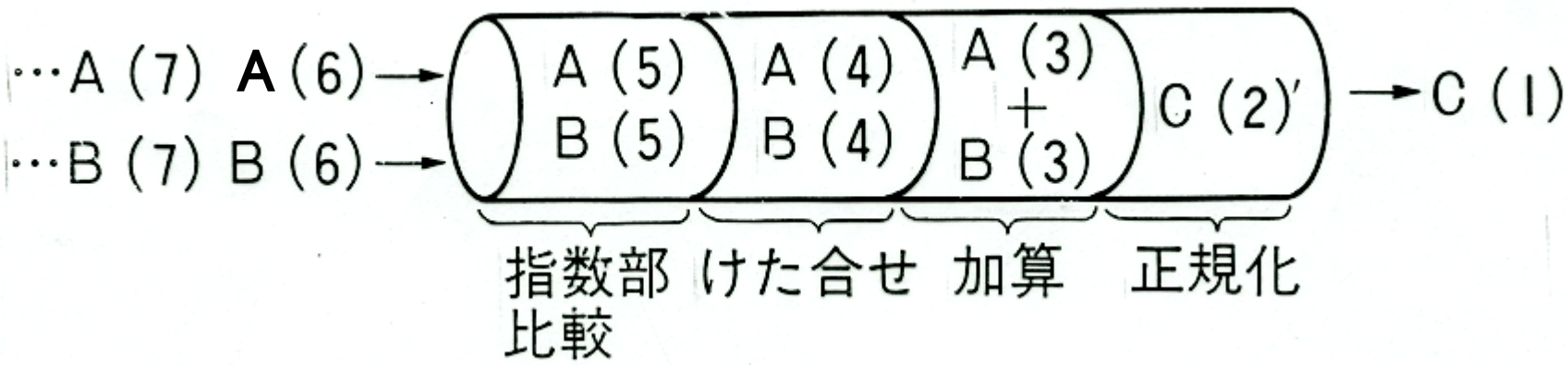
* Million operations per second (mops)

Special hardware features

Random-access memory of 1024 bits per PE
Parity on all array-processor memory
Extra four columns of PEs to allow on-line repairing
Program-controlled edge interconnections
Hardware array resolver to isolate array errors
A buffer memory with corner-turning capability

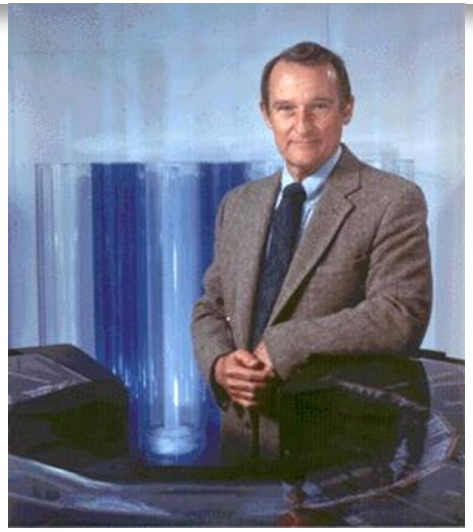
ベクトルパイプライン
Vector Pipeline

Floating Point Addition



Comparison of exponent	Sifting of mantissa	Addition	Normalization

Seymour Cray: Father of Supercomputers using vector pipeline

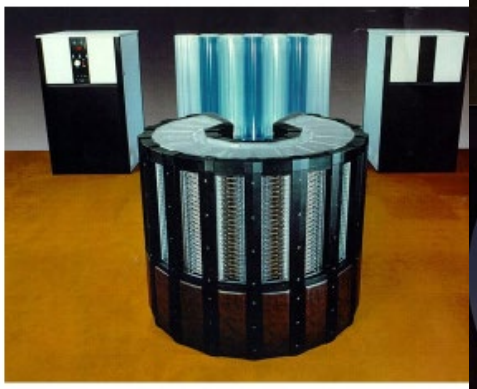


2019 Seymour Cray Award Winner:
David Kirk, NVIDIA Corporation (retired)





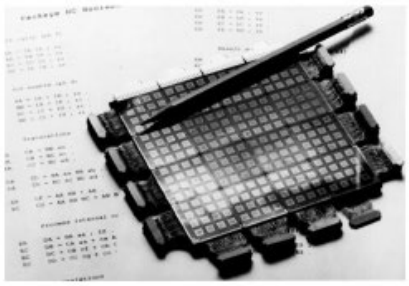
Cray 1



Cray 2



Cray 3





https://www.youtube.com/watch?v=Yc-VFuRWevw

https://sc25.supercomputing.org/program/awards/

SIDNEY FERNBACH MEMORIAL AWARD
Presented at SC 2025



EWA DEELMAN

SEYMOUR CRAY COMPUTER ENGINEERING AWARD
Presented at SC 2025



JOHN M. SHALF

KEN KENNEDY AWARD



SAMAN AMARASINGHE

Sidney Fernbach Award

The Sidney Fernbach Memorial Award is presented annually to an individual for an outstanding contribution in the application of high-performance computers using innovative approaches.
Ewa Deelman, University of Southern California Information Sciences Institute (ISI)

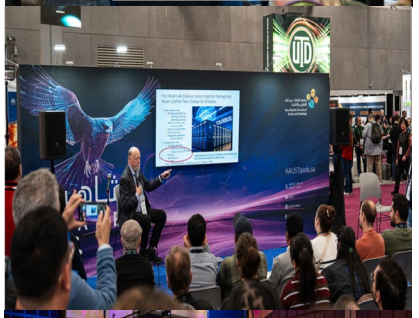
Seymour Cray Computer Engineering Award

The IEEE-CS Seymour Cray Computer Engineering Award recognizes innovative contributions to high performance computing systems that best exemplify the creative spirit demonstrated by Seymour Cray. Recipients may be recognized for technologies that have contributed to working machine deployed at scale, which has had long-term impact valued by the HPC community.
John Shalf, Lawrence Berkeley National Laboratory (LBNL)

Ken Kennedy Award

The ACM/IEEE-CS Ken Kennedy Award recognizes substantial contributions to programmability and productivity in computing and substantial community service or mentoring contributions.
Saman Amarasinghe, Massachusetts Institute of Technology (MIT)






https://scconferencephotos.pixieset.com/sc25stlouisconferencephotos/

Architecture of Fujitsu VP2000

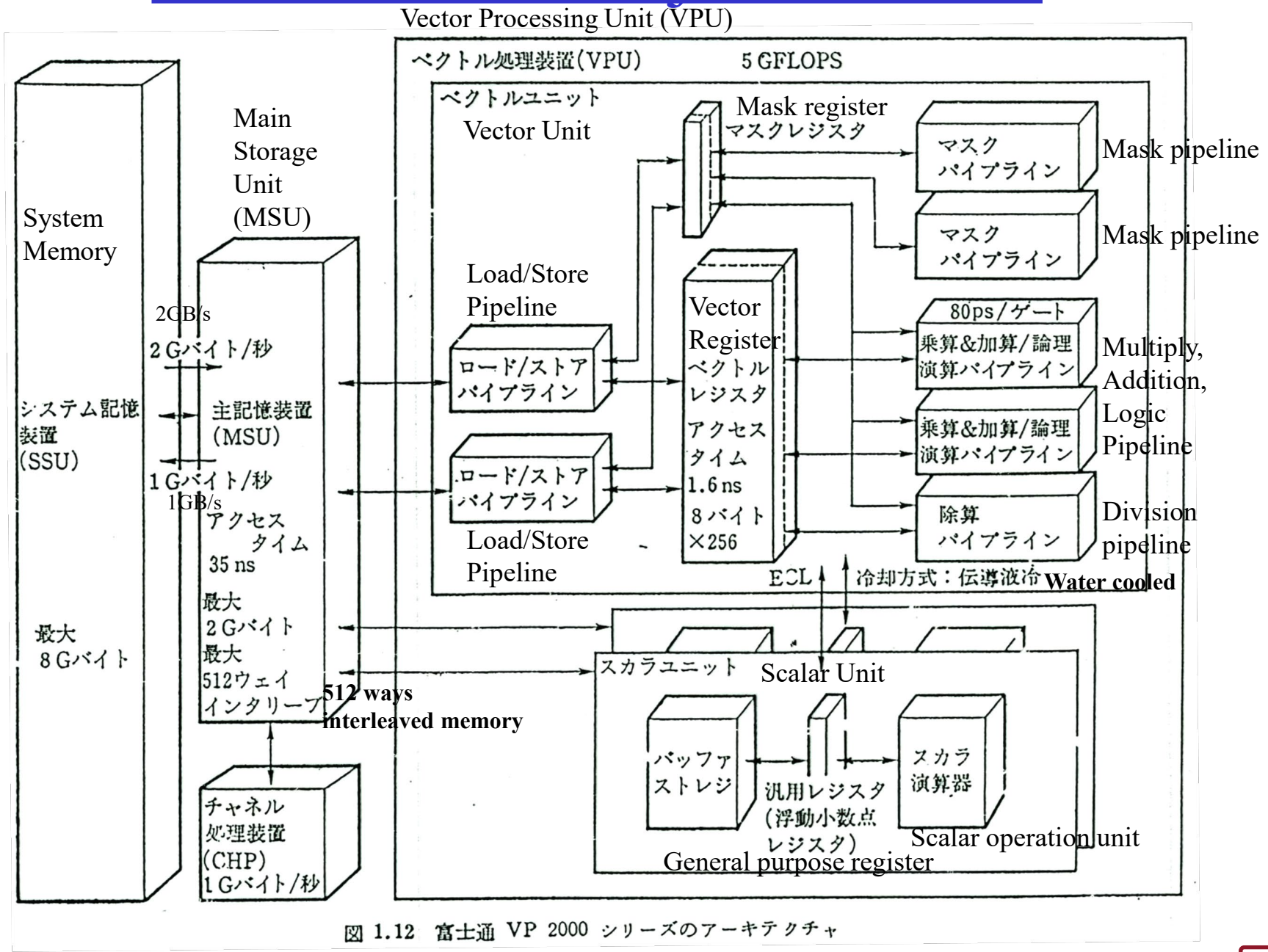
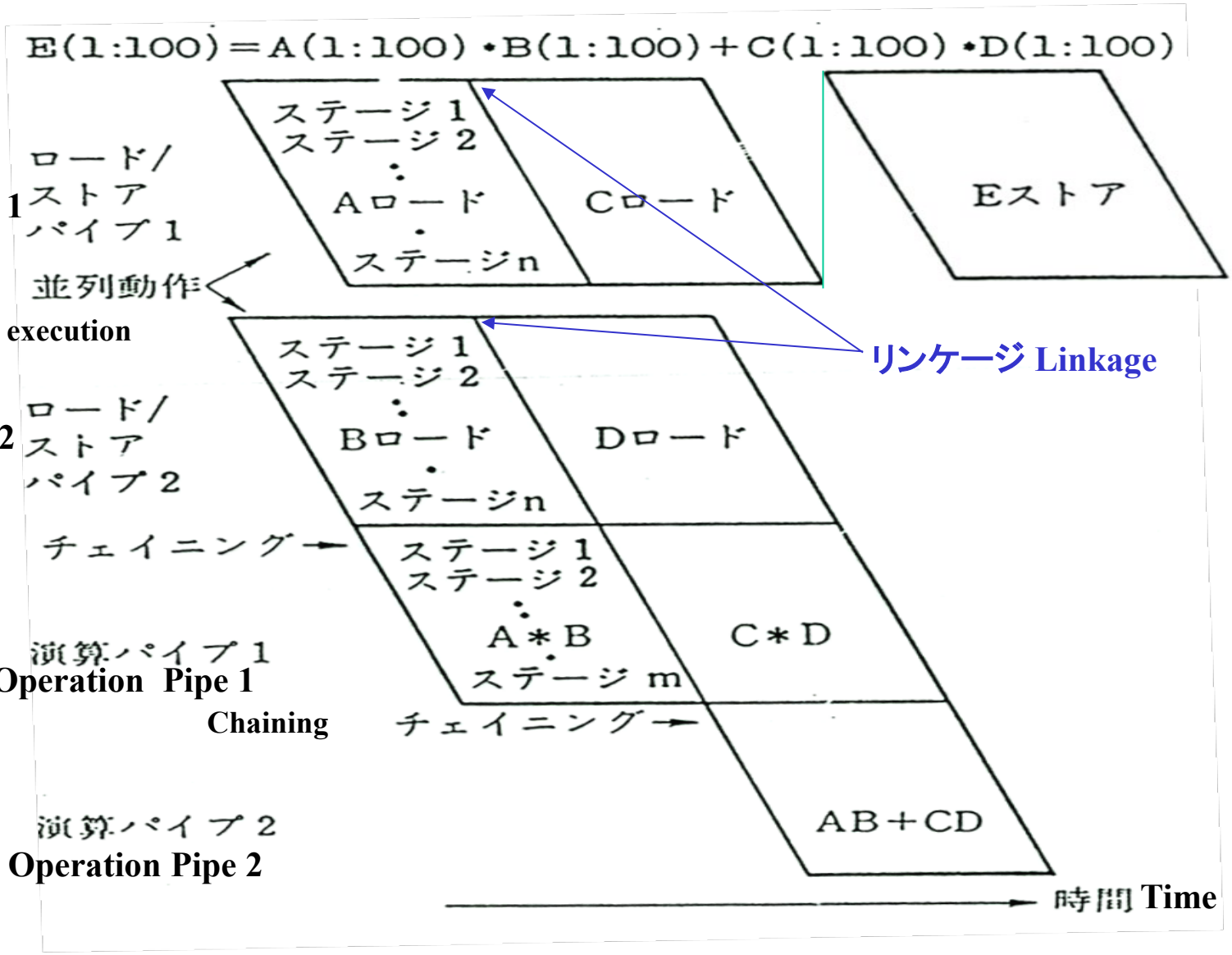


図 1.12 富士通 VP 2000 シリーズのアーキテクチャ

Chaining, Linkage and Parallel Execution in Vector Pipeline Processor



ベクトルパイプラインにおけるチェイニング、リンケージと並列動作

Chaining, Linkage, and Parallel Operations in Vector Pipeline Processor

日本ベクトルスパコンの開発者(Leaders in Japanese Vector Supercomputers)

三好甫氏(早大数学卒)

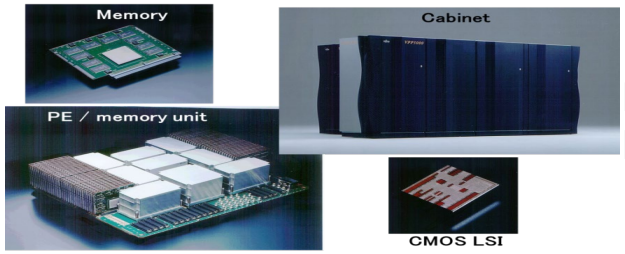


Mr. Hajime Miyoshi
(Waseda Mathematics)

Father of Japanese Supercomputers

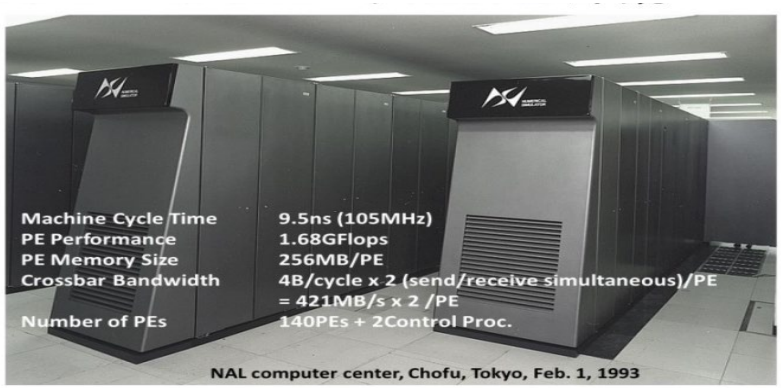


JSPP2000@Waseda スーパーパネルPFLOPSへの道
Super Panel: Road to Peta Flops



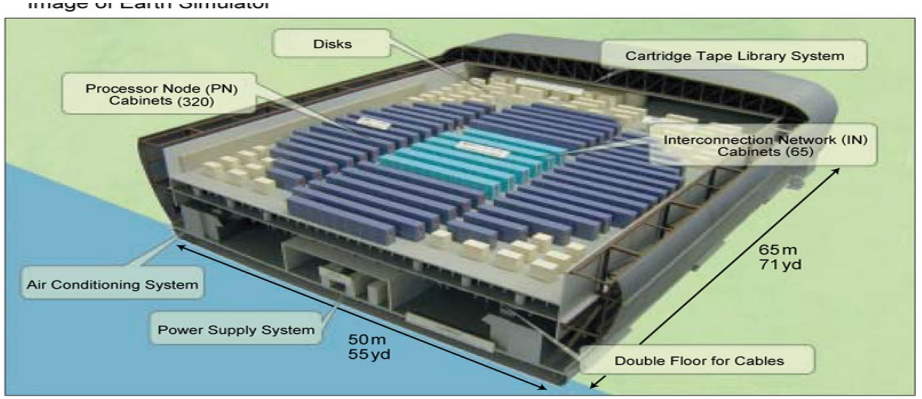
NWTの商用版
Fujitsu VPP5000 (仏気象庁他)

1993年 スーパーコンピュータ数値風洞 (NWT)



2021年ノーベル物理学賞
プリンストン大 真鍋淑郎先生
大気・海洋大循環モデル

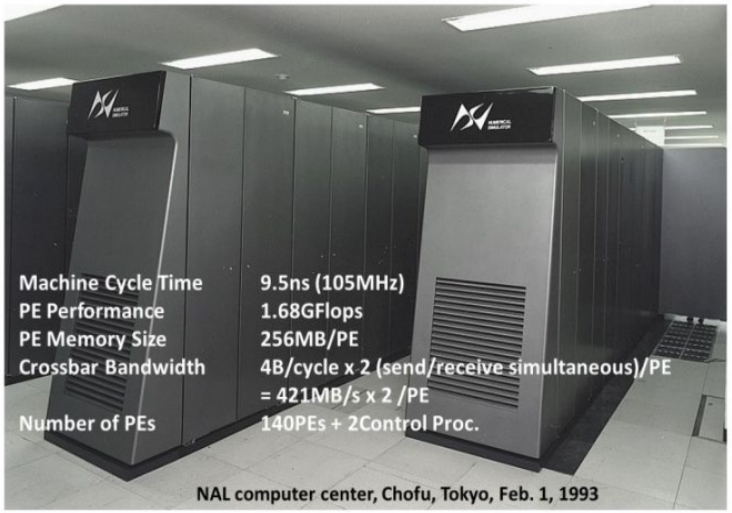
2002年地球シミュレータ
2002 Earth Simulator:
40TFLOPS



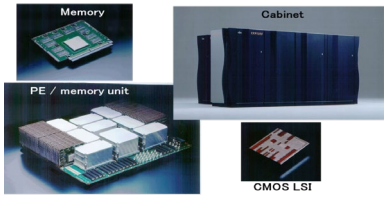
笠原博徳が設計・開発に参加した3つの世界No.1 スーパーコンピュータ “NWT：数値風洞”, “Earth Simulator：地球シミュレータ”, and “K：京”



日本スパコンの父
三好甫氏
航空宇宙技術研究所
早稲田大学数学卒
Father of Japanese Supercomputer
Mr. Hajime Miyoshi,
Waseda Alumnus,
Leader of NWT,
Earth Simulator

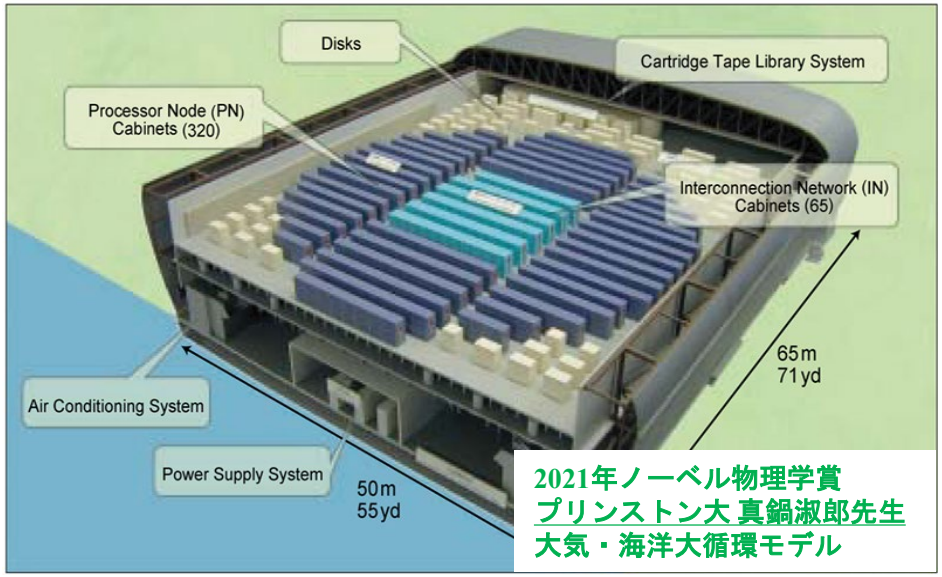


NWT：数値風洞
(Numerical Wind Tunnel) ,
1993,1.68GFLOPS
<Fujitsu VPP 500, 5000>



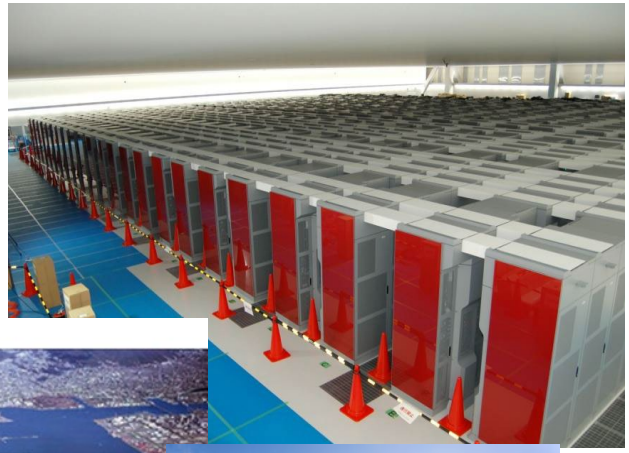
Earth Simulator,
2002

40 TFLOPS Peak ($40*10^{12}$)
35.6 TFLOPS Linpack
3.2MW



2021年ノーベル物理学賞
プリンストン大 真鍋淑郎先生
大気・海洋大循環モデル

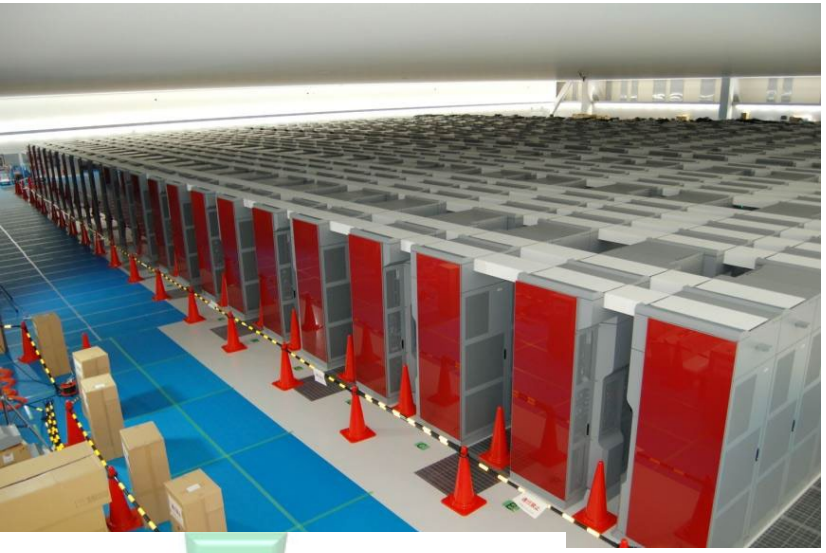
K「京」,
2011
10PFLOPS
11.3MW







“K” Supercomputer by Riken
No.1 in TOP500, June 20 & Nov.2,2011



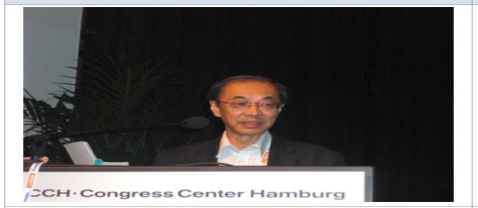




「京」が第37回TOP500ランキングにおいて世界第一位を獲得！
(プレスリリースはこちら)

TOP500リストに登録した「京」のシステムは、現在整備途中段階のもので、672筐体（CPU数68,544個）の構成です。LINPACK（リンパック）ベンチマークでは、世界最高性能の8.162ペタフロップス（毎秒8,162兆回の浮動小数点演算数）を達成し、TOP500リストの首位を獲得しました。また、実行効率は93.0%と高水準の記録を達成しました。
日本のスーパーコンピュータがTOP500リストで第1位となるのは、2004年6月以来のこととなります。

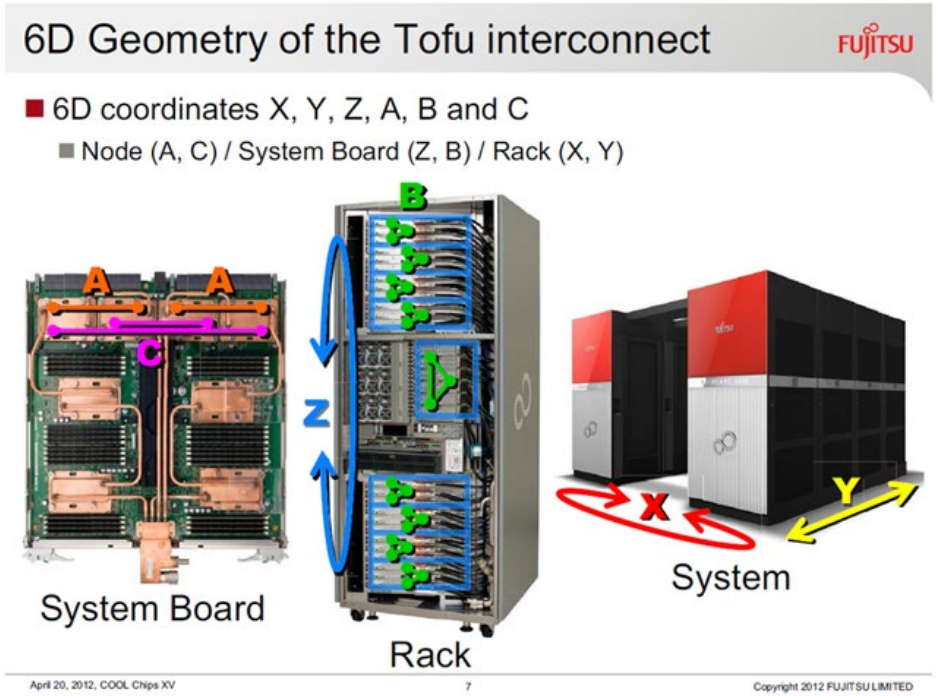
京 K computer

第26回ISC'11における第37回TOP500ランキング認定式	6月20日（月）19：00～ 富士通株式会社との共同記者会見
	
認定式でスピーチをする理研次世代スパコン開発実施本部 渡辺プロジェクトリーダー	富士通株式会社 間塚代表取締役会長（右）と 理研 野依理事長 （左）
	
富士通株式会社 佐相副社長（左）と 理研 渡辺プロジェクトリーダー（中央）	右から、富士通株式会社 井上常務理事、間塚代表取締役会長、 理研 野依理事長、平尾計算科学研究機構長
	
富士通株式会社 佐相副社長（左）と 理研 渡辺プロジェクトリーダー（中央）	右から、富士通株式会社 井上常務理事、間塚代表取締役会長、 理研 野依理事長、平尾計算科学研究機構長



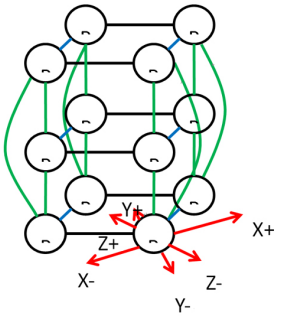
http://www.nsc.riken.jp/K/diary.html

理化学研究所　神戸ポートアイランド　10PFLOPS　京のネットワーク K ‘s network



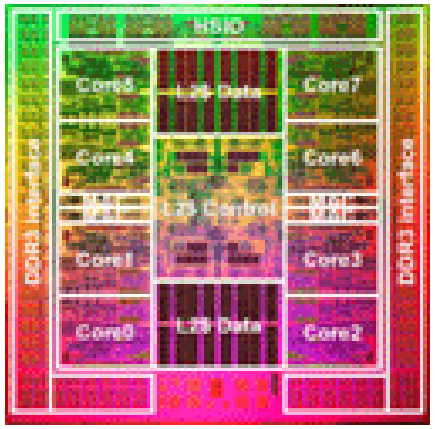
TOFU Network

6D Mesh/ Torus

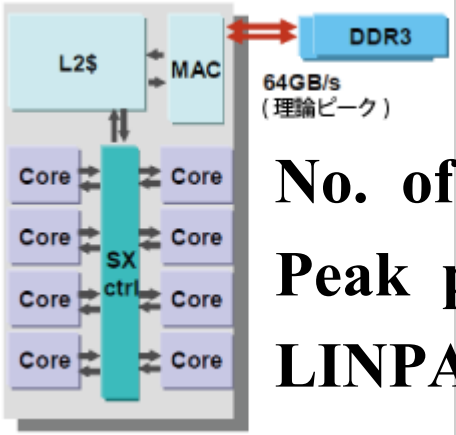




6次元メッシュ/トーラス（概念模型）



SPARC64™ VIIIfx
（提供：富士通(株)）



No. of CPU: 88,128

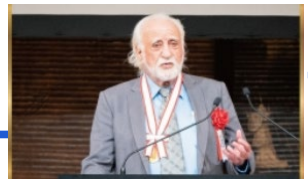
Peak performance: 11.28 PFLOPS

LINPACK performance: 10.51 PFLOPS

Execution Efficiency: 93.2%

Laureates of The Okawa Prize

2025





The Winners of the Okawa Prize 2025

Citation

For pioneering and outstanding research in high-performance microprocessor architectures, especially involving instruction level parallelism, superscalar processor design, and high accuracy branch prediction



Dr. Yale N. Patt

Degree: PhD in Electrical Engineering, Stanford University, 1966

Date of Birth : June 29, 1939

Brief Biography :

1962 BSEE Northeastern University
1963 MS Electrical Engineering, Stanford University
1966 PhD Electrical Engineering, Stanford University
1966-1967
Assistant Professor, Electrical Engineering, Cornell University
1967-1969
United States Army Reserve
1969-1976
Associate Professor, Computer Science, NC State University
1976-1988
Professor of Mathematics and Computer Science, SF State University
1979-1988
Visiting Professor, EECS, UC Berkeley
1988-1999
Professor of EECS, University of Michigan, Ann Arbor
1999-2025
Ernest Cockrell, Jr. Centennial Chair in Engineering, UT Austin

Current Positions and Organization :

1999-present
Professor of Electrical and Computer Engineering, UT Austin
2011-present
University Distinguished Teaching Professor, UT Austin
2025-present
Virginia Cockrell Centennial Chair in Engineering, UT Austin

Main Awards and Honors :

1962 National Tau Beta Pi Fellow
1995 IEEE Emanuel R. Piore Medal
1996 IEEE/ACM Eckert-Mauchly Award
1999 IEEE Wallace W. McDowell Award
2000 ACM Karl V. Karlstrom Outstanding Educator Award
2005 IEEE Charles Babbage Award
2009 Honorary Doctorate, University of Belgrade
2011 IEEE B. Ramakrishna Rau Award (inaugural recipient)
2013 IEEE Harry H. Goode Award
2016 Benjamin Franklin Medal in Computer and Cognitive Sciences,from the Franklin Institute
2007-2024
Eight IEEE/ACM Test of Time Influential Paper Awards
2003-2011
Seven IEEE Micro Top Picks papers
1986-2012
Nine Conference Best Paper Awards
1986-2025
Invited Keynote address at 79 international conferences
1985-2019
Invited Lecturer at 77 University Distinguished Lecture Series
Fellow of the IEEE and ACM
Member of National Academy of Engineering (2014)

Main Achievements :

Yale Patt has amassed an outstanding record of both research and teaching during his 60 years as a professor.

Patt's pioneering research started with his WOS module while he was a graduate student in 1965. He realized that he could implement switching circuits with far fewer logical primitives and increased reliability over single transistor NAND gates in use at the time if he added at incremental cost more transistors to create the logical function performed by the primitive. As a result, he defined the WOS module, a very rich, complex, logically complete primitive as his basic building block, and published a paper at the 1967 Spring Joint Computer Conference.

Patt's most important research achievement came 20 years later with the invention of HPS, his high performance microarchitecture, research he did with three of his PhD students, Wen-mei Hwu, Stephen Melvin, and Michael Shebanow, and published as two papers in MICRO 1985.

HPS combined wide fetch and issue, decoding instructions into micro-ops, and out-of-order execution. Patt added result registers, which other researchers renamed a "reorder buffer" that allowed results to be temporarily stored so they could be retired in program order. In-order retirement guaranteed precise exceptions, retaining the integrity of the ISA.

Patt represented every instruction in a program as a data flow graph consisting of micro-ops. The data flow graph of each instruction was merged into the larger data flow graph consisting of all instructions that had previously been fetched but not yet retired. This established the core as a restricted data flow engine, restricted in the sense that the data flow graph consisted of only instructions that had been decoded and merged, but not yet retired.

Patt also added wide issue fetch and decode of instructions, increasing the rate of micro-ops supplied to the core. The data flow graph allowed the micro-ops themselves to control execution, eliminating the need for a complex but much less effective global controller.

Many dismissed HPS, stating that there was not enough parallelism to warrant the complexity of the engine, and there were not enough transistors available on the chip to implement it. Patt countered that data flow would provide the mechanisms for scheduling micro-ops, and Moore's Law would provide the transistors, which it did! Today, just about all high performance microprocessors embrace the path initiated by HPS.

However, HPS was not done. The microarchitecture was missing the ability to keep the processor core constantly supplied with enough micro-ops. That would require a very aggressive, highly accurate branch predictor – which HPS did not yet have.

Patt and his student Tse-Yu Yeh proposed the Two level dynamic branch predictor in MICRO 1991. Branch prediction had been written off as a concept that would never make a big difference. The branch predictor of choice in 1991 was the saturating 2-bit counter, delivering accurate predictions 80% of the time. That meant taking the wrong path and recovering 20% of the time. The 2-bit counter based its predictions on the history of recent branches. Patt and Yeh claimed it wasn't the recent history but rather what the current branch did the last few times the history was the same as the current history of the branch being predicted. This change increased the prediction accuracy to 95%, decreasing the misprediction rate to 5%, an improvement of approximately 4x over the branch predictors in use at the time. Today, high accuracy branch predictors are an important part of every microprocessor, and follow the concepts introduced by Patt and Yeh.

Patt's research continues to be productive. He and his PhD students have continued to produce seminal research, among them the trace cache with Stephen Melvin (1989) and later with Sanjay Patel (1998), out-of-order fetch with Jared Stark (1999), run ahead execution with Onur Mutlu (2003), diverge-merge processors with Hyesoon Kim (2005), cache replacement with Moin Qureshi (2007), heterogenous processors with Aater Suleman (2010), Morphcore with Khubaib (2012), Tailored Page Sizes with Faruk Guvenilir (2019), Criticality driven fetch with Aniket Deshmukh (2023), and practical Ahead branch prediction with Chester Cai (2024).

For pioneering and outstanding research in high-performance microprocessor architectures, especially involving instruction level parallelism, superscalar processor design, and high accuracy branch prediction, Yale N. Patt is hereby awarded the Okawa Prize.



Tadashi Watanabe

Citation

Pioneering and significant contributions to the development of the high performance computers, including the world's fastest supercomputers, “Earth Simulator" and "K computer”

現　職　理化学研究所　名誉研究

生年月日　1944年3月19日

学　位　博士(情報科学)2005年東北大学

略　歴
1962年　東京大学工学部電子工学科 卒業
1968年　東京大学大学院
工学系研究科修士課程
電気工学専攻 修了
1968年　日本電気(株)入社
1982年　同 コンピュータ技術本部技術課長
1986年　同 コンピュータ技術本部第4技術部長
1988年　同 コンピュータ技術本部長代理
1990年　同 スーパーコンピュータ販売推進本部長
1999年　同 NECソリューションズ支配人
1999年～2005年
(財)高度情報科学技術研究機構 理事
2006年　文部科学省研究振興局研究振興官
2006年　理化学研究所(理研)
次世代スーパーコンピュータ
開発実施本部プロジェクトリーダー
2012年　理研　特別顧問
2013年　理研　名誉研究員

受　賞
1998年　ACM/IEEE Eckert-Mauchly Award
2006年　IEEE Seymour Cray Computer Engineering賞
2006年　IEEE Fellow
2007年　日本計算工学会功績賞
2008年　米国工学アカデミー(NAE)外国会員
2009年　日本学士院賞(矢川元基博士と共同受賞)

主な業績

渡邊貞博士は、1968年日本電気株式会社（NEC）に入社、汎用大型計算機ACOS-6アーキテクチャ設計、犯罪捜査等で使用された指紋照合装置（FMP）やAI用LISPプロセッサの開発に参加後、1982年４月から、ＮＥＣスーパーコンピュータ開発プロジェクトにてプロジェクト管理、システム設計、アーキテクチャ設計を担当され、1985年にGFLOPSの壁を破ったSX-1/SX-2システムの製品化に成功した。1989年から、同博士はスーパーコンピュータの製品計画、販売支援を統括すると共に海外のスーパーコンピュータ事業を統括し、世界の研究機関・大学計算センター等との交流を深め、ユーザの要求や期待に応えるスーパーコンピュータの開発をリードされた。

このＮＥＣ ＳＸシリーズは、国内外の大学の計算センター・研究所、世界各国の気象センターや航空宇宙研究所、自動車メーカーの車の開発等で科学技術研究開発基盤として利用され、世界の基礎科学研究の進展、自動車・飛行機等産業技術開発、天気予報等の人々の生活の向上に貢献した。このSXスーパーコンピュータシリーズの開発並びに世界への貢献が評価され、1998年に、世界の2大コンピュータ学会であるACM（Association for Computing Machinery）とIEEE CS(Computer Society)より、ENIAC開発者の名前を冠したACM/IEEE Eckert-Mauchly賞が授与された。

1997年よりは、科学技術庁宇宙開発事業団及び動力

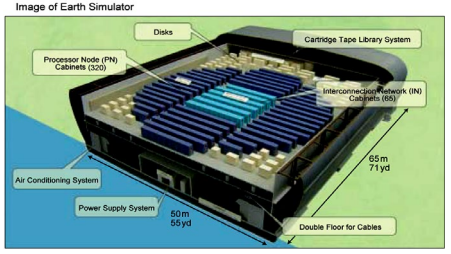
レータ開発」が故　二好甫氏をリーダとしてスタートし、渡邊博士はNECによるシステム設計及び製造をリードされた。地球シミュレータは、2002年２月に完成し、地球温暖化シミュレーションのための大気海洋大循環モデルにおいて５TFLOPSの目標性能を大幅に上回る16TFLOPS、ピーク性能40TFLOPSを達成し、６月にはスパコンランキング「TOP500」Top1に認定され、11月のACM/IEEE主催のスーパーコンピュータ国際会議SC2002　Top500ランキングでも、LINPACK性能35.86TFLOPSと、2位の米国ASCI-Qの7.73TFLOPSを大きく上回る驚異的な性能値で、世界1位となり、New York Times誌に、スプートニク（Sputnik）ショックにも匹敵するコンピュートニク(Computenik)ショックとの記事が掲載される偉業を達成した。このシステムは、2004年6月まで長期にわたり世界1位に留まり、2018年のSC30周年記念の歴代General Chairパネル討論でも30年間で最も記憶に残ったマシンの一つと評された。地球シミュレータは、主として、気候変動予測や地殻変動など地球規模の環境予測に使用され、2007年にノーベル平和賞を受賞された『気候変動に関する政府間パネル（IPCC）』にも大きな貢献を行った。これらの成果により、同博士は、スーパーコンピュータ・アーキテクトの最高栄誉であるIEEE Seymour Cray賞をSC2006にて受賞されている。

また、2006年には、2012年度10PFLOPS（1京回浮動小数点演算毎秒）の実現を目指した文部科学省スーパーコンピュータ「京」開発プロジェクトがスタートし、同博士は理化学研究所にプロジェクトリーダーとして移籍した。

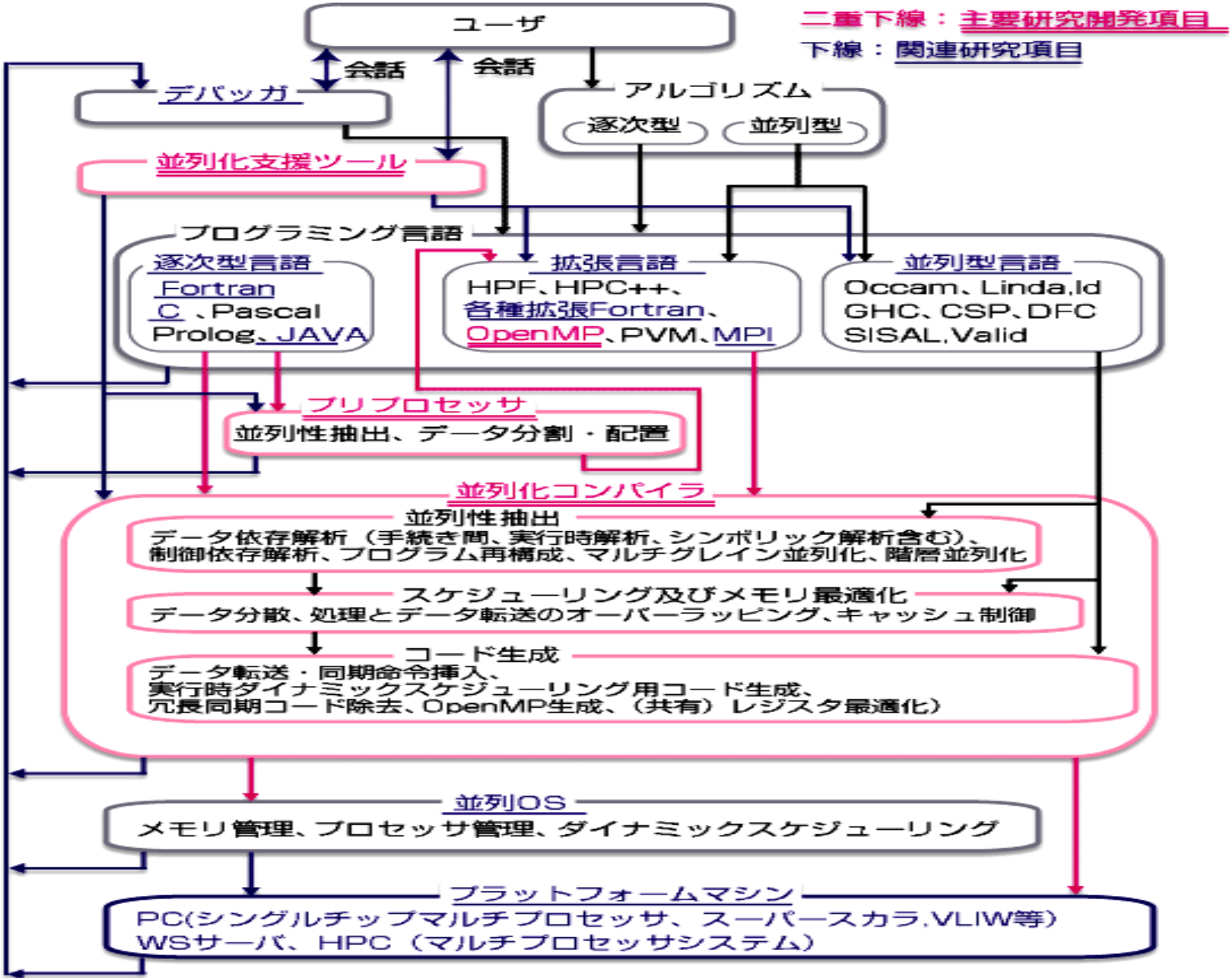
京は、開発メンバーの懸命な努力による開発の前倒しにより、2011年6月のTop500ランキングで、LINPACK性能8.16PFLOPSで世界トップとなり、2011年8月までに全システムの設置が完了し、同年11月のSC11で10.51PFLOPS(1.051京回浮動小数点演算毎秒)と当初目標を達成し、Top 1を維持できた。京は、2012年の稼働開始後、防災、宇宙、気象、創薬・医療、材料科学等広い分野で使用され、科学技術の発展に寄与した。

渡邊博士はNECに入社以来一貫してコンピュータ開発に携わり、特にスーパーコンピュータでは、国産スーパーコンピュータSXシリーズの開発、地球シミュレータ・京等、世界最速スーパーコンピュータの開発を通し、世界の計算科学の発展、自動車・航空機・半導体を含めた最先端技術の発展、地球温暖化予測を含めたSDGsへの貢献等多大なる成果を上げられた。ここに「大川賞」を贈呈し、その功績をたたえるものである。

Image of Earth Simulator



アドバンスト並列化コンパイラプロジェクト



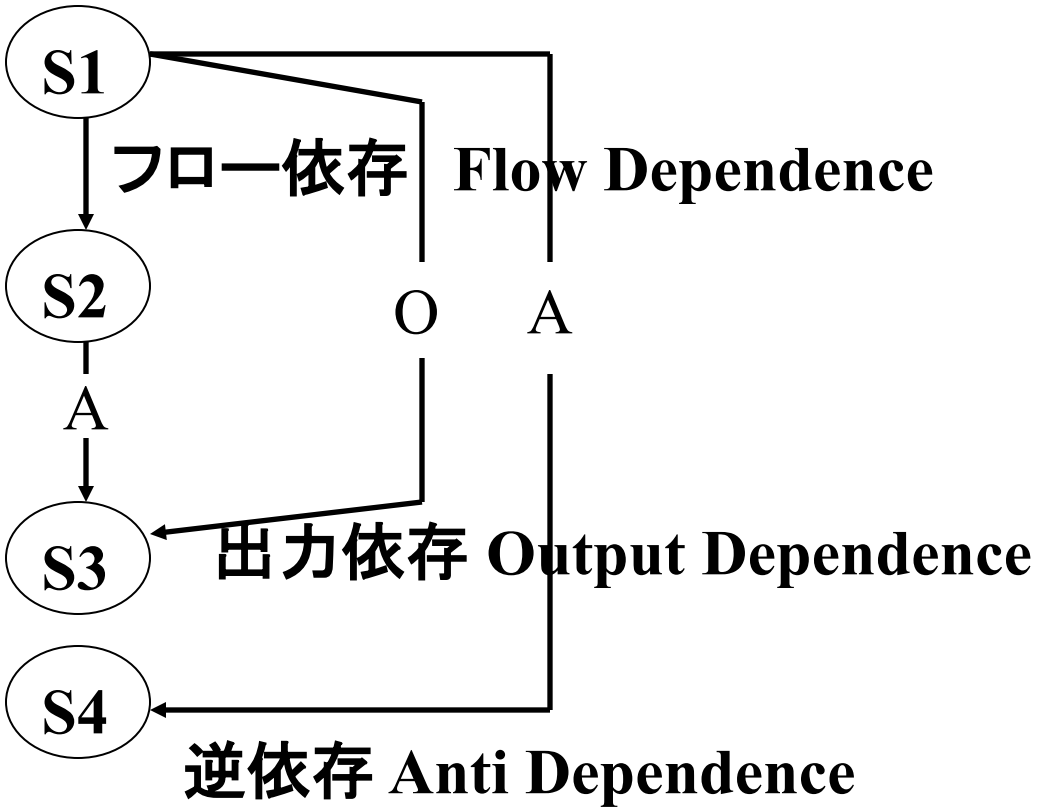
データ依存　Data　Dependency

A=B+C : S1

D=A+E : S2

A=C+F : S3

B=E+F : S4



参考書

笠原博徳「並列処理技術」，コロナ社

Reference

Hironori Kasahara, Parallel Processing Technology, Corona Publisher

自動ベクトル化　Automatic Vectorization

最内側ループのLoop Distribution及びベクトル化

Loop distribution and vectorization of the inner-most loop

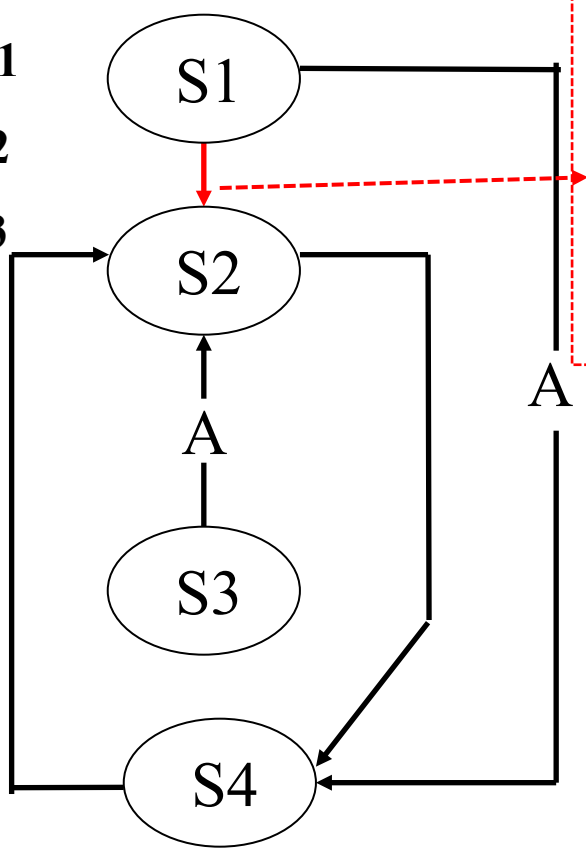
```
do I = 2,N
  C(I)=A(I)+B(I)      :S1
  D(I)=C(I)+A(I-1)    :S2
  E(I)=D(I+1)+X       :S3
  A(I)=D(I) * B(I)    :S4
end do
```

(a) Doループの例

A Do loop

(b)　(a)のデータ依存グラフ

Data dependence graph



エッジが下向きの時はループ分散可能

Loop Distribution is applicable to downward edges

```
do I = 2,N
    C(I)=A(I)+B(I)      :S1
end do
do I = 2,N
   D(I)=C(I)+A(I-1)   :S2
    ・・・
```

(c) ベクトル化されたコード

Vectorized Code

```
C(2:N)=A(2:N)+B(2:N)    :S1
E(2:N)=D(3:N+1)+X       :S3
do I=2,N
     D(I)=C(I)+A(I-1)       :S2
     A(I)=D(I)*B(I)         :S4
end do
```

データ依存解析 Data Dependence Analysis (DA)

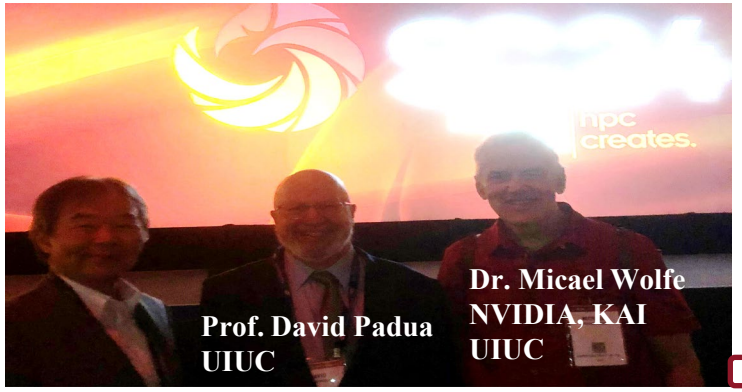
並列性解析：フロー依存、出力依存、逆依存

For parallelization: Flow, Output, Anti dependences

データの分割配置：入力依存

For data locality Optimization: Input dependence

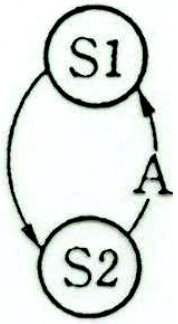
- シンボリックデータ依存解析: **Symbolic DA**
- セマンティックデータ依存解析 **Semantic DA**
- イタレーションスペースデータ依存解析**Iteration space DA**
- 依存方向ベクタ解析 **Dependence vector DA**
- GCD (Greatest Common Devisors)テスト **GCD test**
- **Banerjee's exact test**
- **Banerjee-Wolfe's inexact test**
- **Pugh's OMEGA test**



ノード スプリッティング

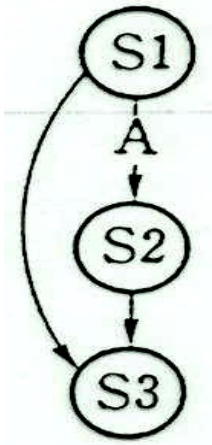
```
doI=1,N
  C(I)=A(I)+B(I)     :S1
  D(I)=C(I)+C(I+1):S2
end do
```

(a)




```
doI=1,N
  TEMP(I)=C(I+1)       :S1
  C(I)=A(I)+B(I)       :S2
  D(I)=C(I)+TEMP(I)  :S3
end do
```

(b)



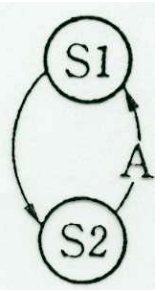

```
TEMP(1:N)=C(2:N+1)
C(1:N)=A(1:N)+B(1:N)
D(1:N)=C(1:N)+TEMP(1:N)
```

(c)

図 3.7　ノード スプリッティング

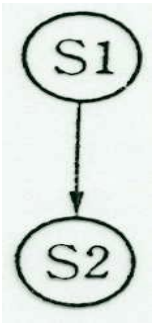
スカラ エクスパンション

```
Do I=1,N
   T=A(I)*B(I)       :S1
   C(I)=T+B(I)       :S2
end do
```



(a)

```
Do I=1,N
   TEMP(I)=A(I)*B(I)    :S1
   C(I)=TEMP(I)+B(I)    :S2
end do
T=TEMP(N)
```



(b)

```
TEMP(1:N)=A(1:N)*B(1:N)
C(I:N)=TEMP(1:N)+B(1:N)
T=TEMP(N)
```

(c)

図 3.8　スカラ エクスパンション

プログラム・リストラクチャリング
Program Restructuring

ベクトル化あるいはループ並列化できないループの自動変換

Automatic code change for non-parallelizable or –vectorizable loops

(a)ステートメントの実行順序の変更: Change of statement execution order

(b)ループディストリビューション: Loop distribution

(c)ノードスプリッティング、スカラエクスパンション:
Node splitting, Scaler expansion

(d)ループインターチェンジ、パーミュテーション:
Loop Inter-change, Permutation

(e)ループアンローリング: Loop unrolling

(f)ストリップマイニング: Strip-mining

(g)アレイプライベタイゼーション: Array privatization

(h)ユニモジュラー変換: Uni-modular transform

（ループリバーサル、パーミュテーション、スキューイング
Loop reversal, Permutation, Skewing by Affine Transformation）

世界の並列化コンパイラ・トップ研究者

（LCPC 2012, 25th Languages and Compilers for Parallel Computing,
早稲田大学グリーンコンピューティングセンター開催時））

